

COMPSCI 701:
Creating Maintainable Software
Lecture 9.2: SOLID and Alterability II

Ewan Tempero, Yu-Cheng Tu
School of Computer Science

Agenda

- Agenda

- ISP
- DIP
- Class Exercise
- SOLID Summary
- Key Points

- Admin
 - ?
- ISP
- DIP
- Class exercise

- Agenda
- **ISP**
- DIP
- Class Exercise
- SOLID Summary
- Key Points

Interface Segregation Principle

- Make fine-grained interfaces that are client specific
- Or, clients should not be forced to depend on methods that they do not use
 - if those methods change, the clients may need to change unnecessarily
- Generally, SRP should result in classes whose clients need (in principle) all methods provided, but sometimes that's not possible
 - Multiple distinct concepts that belong to a more general category and some clients only care about the category (e.g. savings account, cheque account, term deposit — all “bank accounts”)
 - legacy code
 - implementation requirements mean different responsibilities in the same implementation (class)

- Agenda
- **ISP**
- DIP
- Class Exercise
- SOLID Summary
- Key Points

Example

```
public class Artist implements Comparable<Artist> {  
    private String _name;  
    public Artist(String name) { ... }  
  
    public String getName() { ... }  
  
    public int compareTo(Artist artist) {  
        if (artist == null) {  
            return 1;  
        }  
        return _name.compareTo(artist._name);  
    }  
}
```

```
public class Date implements Comparable<Date> {  
    private int _year;  
    private int _month;  
    private int _day;  
  
    public Date(String iso8601Date) throws InventoryManException {...}  
    public String toString() { ... }  
    public int compareTo(Date o) { ... }  
}
```

- Clients can require `Artist` and `Date`, but if all they need to do is compare `Artist` or `Date` objects (e.g. sorting algorithms) then they don't need to depend on the fact that they are `Artist` or `Date`
- `Comparable` provides a fine-grained interface for those clients that only care about the ability to compare objects

ISP and Alterability

- Agenda
- **ISP**
- DIP
- Class Exercise
- SOLID Summary
- Key Points

- A client that depends on a method it does not use may then have to change when that method changes—it has an unnecessary dependency on that method. Removing the dependency reduces the likelihood the client needs to change $\Rightarrow$ reducing C.

ISP and Design Patterns

- Agenda
- **ISP**
- DIP
- Class Exercise
- SOLID Summary
- Key Points

- Many design patterns define small interfaces that are used in the pattern, but the actual classes will have larger interfaces
- Observer — the observer classes will usually have many more methods than `update()`

- Agenda
- **ISP**
- DIP
- Class Exercise
- SOLID Summary
- Key Points

ISP Example: JavaParser — ClassOrInterfaceDeclaration

OVERVIEW
PACKAGE
CLASS
USE
TREE
DEPRECATED
INDEX
HELP

SUMMARY: NESTED | FIELD | CONSTR | METHOD
DETAIL: FIELD | CONSTR | METHOD

SEARCH:

Package com.github.javaparser.ast.body

Class ClassOrInterfaceDeclaration

```

java.lang.Object
  com.github.javaparser.ast.Node
    com.github.javaparser.ast.body.BodyDeclaration<T>
      com.github.javaparser.ast.body.TypeDeclaration<ClassOrInterfaceDeclaration>
        com.github.javaparser.ast.body.ClassOrInterfaceDeclaration

```

All Implemented Interfaces:

```

NodeWithAbstractModifier<ClassOrInterfaceDeclaration>,
NodeWithAccessModifiers<ClassOrInterfaceDeclaration>,
NodeWithFinalModifier<ClassOrInterfaceDeclaration>,
NodeWithPrivateModifier<ClassOrInterfaceDeclaration>,
NodeWithProtectedModifier<ClassOrInterfaceDeclaration>,
NodeWithPublicModifier<ClassOrInterfaceDeclaration>,
NodeWithStaticModifier<ClassOrInterfaceDeclaration>,
NodeWithStrictfpModifier<ClassOrInterfaceDeclaration>,
NodeWithAnnotations<ClassOrInterfaceDeclaration>,
NodeWithExtends<ClassOrInterfaceDeclaration>,
NodeWithImplements<ClassOrInterfaceDeclaration>,
NodeWithJavadoc<ClassOrInterfaceDeclaration>,
NodeWithMembers<ClassOrInterfaceDeclaration>,
NodeWithModifiers<ClassOrInterfaceDeclaration>, NodeWithRange<Node>,
NodeWithSimpleName<ClassOrInterfaceDeclaration>, NodeWithTokenRange<Node>,
NodeWithTypeParameters<ClassOrInterfaceDeclaration>, Observable, Visitable,
HasParentNode<Node>, Resolvable<ResolvedReferenceTypeDeclaration>, Cloneable

```

- 23 smaller interfaces for the same class! (arguably +4 more)

- Agenda
- **ISP**
- DIP
- Class Exercise
- SOLID Summary
- Key Points

ISP Conclusion

- Many presentations (including Martin's) make it clear it is multiple interfaces for the **same class**
 - *but many on-line tutorials focus on — incorrectly — creating classes with “small” interfaces*
- In principle, actionable—identify the set of methods actually used in a class and create an interface that exactly matches that set
- Needs care as can lead to many tiny interfaces that are so abstract that they have lost any useful meaning

```
public interface Stringable {  
    String toString();  
}
```

(But see JavaParser example) (And role interfaces
<https://martinfowler.com/bliki/RoleInterface.html>)

- Agenda
- ISP
- **DIP**
- Class Exercise
- SOLID Summary
- Key Points

The Dependency Inversion Principle

- The Dependency Inversion Principle, Robert C. Martin, C++ Report, May 1996
 - “High level modules should not depend upon low level modules. Both should depend upon abstractions”
 - “Abstractions should not depend upon details. Details should depend upon abstractions”
- Hollywood principle — “don’t call us, we’ll call you”
- “inversion of control”

- Agenda
- ISP
- **DIP**
- Class Exercise
- SOLID Summary
- Key Points

Inversion of control

- “typical” call sequence — code A calls code B, and control returns to code A
- “inversion” — code A is **passed** to code B. B calls code A, and control returns to B

```
public interface FileOperation {
    void execute(Browser browser);
}
public class ListFiles implements FileOperation {
    public void execute(Browser browser) {
        browser.printFile();
    }
}
```

```
public class Browser {
    private void browse() {
        ...
        while (true) {
            String cmd = getUserOperation();
            FileOperation op = null;
            if (cmd.equals("L")) {
                op = new ListFiles();
                ....
                op.execute(this);
            }
        }
        ...
        public void printFile() {
            ...
        }
    }
}
```

Depend on Abstractions

- Agenda
- ISP
- **DIP**
- Class Exercise
- SOLID Summary
- Key Points

```
interface MovieFinder {
    List<Movie> findAll();
}

class MovieLister {
    // MovieLister depends on abstract MovieFinder
    // not concrete CVSMovieFinder
    private MovieFinder _finder = null;

    public MovieLister(MovieFinder finder) {
        _finder = finder;
    }

    public List<Movie> moviesDirectedBy(String dir) {
        List<Movie> allMovies = _finder.findAll();
        // iterate through 'allMovies' and find those with
        // 'dir' as director...
    }
}
```

DIP and Alterability

- Agenda
- ISP
- **DIP**
- Class Exercise
- SOLID Summary
- Key Points

- A class that depends on concrete types is more likely to need to change if any of the concrete types change than if it depends on abstract types $\Rightarrow$ reduces C

DIP and Design Patterns

- Agenda
- ISP
- **DIP**
- Class Exercise
- SOLID Summary
- Key Points

- **Dependency Injection** provides the means to follow the DIP, means that the client depends on the abstract service, not the concrete service that is injected
- Strategy Pattern — Context depends on abstract strategy

- Agenda
- ISP
- **DIP**
- Class Exercise
- SOLID Summary
- Key Points

DIP Conclusions

- Actionable — look for opportunities to use abstractions rather than concretations
- Some uncertainty about primitive types, and don't always have abstractions available (e.g. third party libraries)

- Agenda
- ISP
- DIP
- **Class Exercise**
- SOLID Summary
- Key Points

Class Exercise

- Let's look at another version of the Noughts and Crosses, and answer the following:
 - How does this implementation follow LSP? Don't forget to include class names and line numbers.

SOLID Summary

- Agenda
- ISP
- DIP
- Class Exercise
- **SOLID Summary**
- Key Points

- Martin is a well-known and respected commentator on object-oriented design
- Much of what he (and others) say about SOLID isn't new, but captures in a useful way some principles that should be better known than they are
- Some principles are hard to follow due to subjective nature (e.g. SRP)
- For some principles, their use can be measured (DIP, LSP)
- The principles are not independent: DIP is only likely to work well when SRP has been followed with respect to the type being injected and LSP is used for all child types of the injected type. OCP also only likely gives the most benefit with SRP has been followed. ISP really assumes LSP. (etc)
- All exploit **polymorphism**
- But (as always) the cost/benefit trade-off is difficult to determine
 - “Following SRP requires adding more classes, which is a cost (creating, testing, maintaining). What is this cost, and what benefit do I get by doing so?”
- The arguments given as to why the SOLID principles are useful are consistent with our view of alterability

- Agenda
- ISP
- DIP
- Class Exercise
- SOLID Summary
- Key Points

Key Points

- The SOLID principles are considered important to good design by many in industry, and so should be understood if only because you are expected to
- Some of the principles have subtleties that are not well described, the principles are not actionable $\Rightarrow$ cannot just base argument on literal interpretation
- Many of the examples given are “best guesses” due to the lack of objective definitions for some principles